



Answer for **Top 10 Angular Interview Questions**

Top 10 Angular **Interview Questions** **You Must Be Ready For!**

Concepts that separate
“Angular users”  from “Angular experts” 

→ **Swipe**  to learn all 10
Save this post  **Share with friends** 

By **Pankaj Ikhhar**

Solution Architect | Mentor | Coach



Follow for Career Growth & Interview Preparation

Q1 AngularJS vs Angular

✓ **What really changed?**
And why it matters?

✓ **AngularJS (1.x)** >>  << ✓ **Angular (2+)**

- ✓ JavaScript + \$scope based
- ✓ MVC pattern
- ✓ Digest cycle (watchers) → slow in large apps
- ✓ No TypeScript by default
- ✓ Harder to scale enterprise projects

- ✓ TypeScript + Component architecture
- ✓ RxJS for async programming
- ✓ Better performance (Ivy, AoT)
- ✓ CLI + strong tooling
- ✓ Built for scalable enterprise apps

 **Interview line:**

Angular is a complete rewrite focused on performance + architecture + scalability.

By **Pankaj Ikhar**

Solution Architect | Mentor | Coach

 Follow for Career Growth & Interview Preparation

Q2 🚀 Core Building Blocks

✓ What are the core building blocks of Angular?



✓ Components

- ✓ Main UI building block (HTML + TS + CSS)
- ✓ Contains **template** + business logic
- ✓ **Reusable** and testable

Example: UserComponent, DashboardComponent



✓ Directives

- ✓ **Structural** → changes DOM structure
→ *ngIf, *ngFor
- ✓ **Attribute** → changes look/behavior
→ [ngClass], [ngStyle]



Modules (NgModule)

- ✓ Groups components/services together
- ✓ Helps **organize** features (legacy style)

📍 Modern Angular supports **Standalone Components** (less module dependency)



✓ Services

- ✓ Shared reusable logic (API calls, state, utilities)
- ✓ Used with **Dependency Injection** (DI)

🔥 **Interview line:** Angular apps are built using **Components**, powered by **Services**, enhanced by **Directives**, and organized using **Modules / Standalone structure**.

By **Pankaj Ikhar**

Solution Architect | Mentor | Coach



Follow for Career Growth & Interview Preparation

Q3 🚀 Data Binding

✅ How does Data Binding work in Angular?

Interpolation

- ✓ Used to display values inside HTML

```
{{ userName }}
```

- ✓ Best for text rendering

Property Binding

- ✓ Bind TS value → HTML property

```
[disabled] = "isLoading"
```

- ✓ Best for DOM properties like src, value, checked

Event Binding

- ✓ Bind UI events → trigger TS function

```
(click) = "save()"
```

- ✓ Best for buttons, inputs, key events

Two-way Binding

- ✓ Data sync both ways (UI ↔ TS)

```
[(ngModel)] = "email"
```

- ✓ Best for form inputs

Interview line:

- ✓ Property Binding: TS → UI
- ✓ Event Binding: UI → TS
- ✓ Two-way Binding: both directions

By **Pankaj Ikhar**

Solution Architect | Mentor | Coach

 Follow for Career Growth & Interview Preparation

Q4 🚀 *ngIf vs [hidden]

✅ What is the difference between *ngIf and [hidden]?

★ *ngIf

- ✅ Adds / removes element from DOM
- ✅ Component gets created & destroyed
- ✅ Better memory usage when hidden
- ✅ Best for heavy UI blocks (tables, forms, components)
- 🔥 Lifecycle runs when recreated (ngOnInit again)

Example:

```
<div *ngIf=isLoggedIn Dashboard</div>
```

🏠 [hidden]

- ✅ Element stays in DOM
- ✅ Only toggles visibility using CSS (display:none)
- ✅ Good for fast UI toggle
- ❌ DOM + bindings still exist → memory used
- ❌ Component state stays in memory

Example:

```
<div [hidden]=!isLoggedIn Dashboard</div>
```



Interview line:

- ✅ Use *ngIf for true conditional rendering
- ✅ Use [hidden] when you need frequent show/hide without destroying DOM.

By Pankaj Ikhar

Solution Architect | Mentor | Coach

📍 Follow for Career Growth & Interview Preparation

Q5 🚀 Dependency Injection (DI)

✅ How does **Dependency Injection** work in Angular?

✅ What is DI?

Dependency Injection means Angular creates and **provides objects** automatically, instead of you doing `new MyService()` everywhere.

✅ Benefits:

- ✓ Loose coupling
- ✓ Reusability
- ✓ Easy unit testing
- ✓ Cleaner architecture

↪ **with DI**

🔹 `UserService` → `=` `UserComponent`

✅ Hierarchical Injectors (Very Important 🔥)

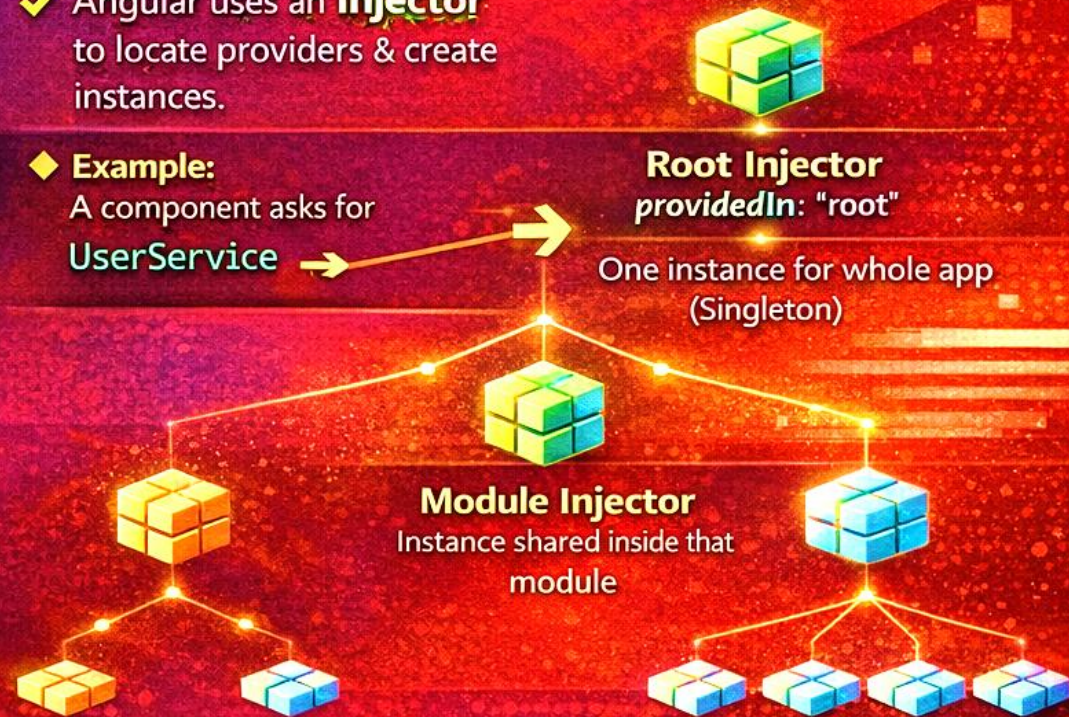
- ✓ Angular uses an **Injector** to locate providers & create instances.

◆ Example:

A component asks for `UserService` →

Root Injector
providedIn: "root"

One instance for whole app
(Singleton)



🔥 Interview line:

- ✅ Angular DI is **hierarchical** → nearest provider wins (`root` → `module` → `component`)

By Pankaj Ikhar

Solution Architect | Mentor | Coach

📍 Follow for Career Growth & Interview Preparation

Q6 🚀 RxJS Basics

✅ Observable vs Subject vs BehaviorSubject What's the difference?

✅ Observable

- ✓ A data stream over time
- ✓ You can subscribe to receive values
- ✓ No manual emit from outside

📌 Common in Angular:
HttpClient.get() returns
Observable

- 📌 Example use-case:
- API calls, async data fetch



✅ Subject

- ✓ Observable + Observer
- ✓ Means: it can emit values using **.next()**
- ✓ Useful when **multiple** subscribers need same value (multicast)

📌 Example use-case:
→ shared events, UI triggers



✅ BehaviorSubject

- ✓ A Subject with latest value stored
- ✓ Requires an initial value
- ✓ New subscriber instantly gets **current / latest value**

📌 Example:
→ login state, user profile, app-wide state management



🔥 **Interview line:** ✅ Observable = producer controlled stream

✅ Subject = you can push values anytime

✅ BehaviorSubject = Subject + remembers last value

By Pankaj Ikhar

Solution Architect | Mentor | Coach

📍 Follow for Career Growth & Interview Preparation

Q7 🚀 Routing & Guards

✅ What are Route Guards in Angular? And why do we use them?

✅ What is Angular Routing?

Routing helps navigate between pages/components using URLs.

Example: `/login → /dashboard → /profile`

✅ What are Route Guards?

Guards are used to **control access** to routes before navigation happens.

✅ Used for:

- ✓ **Authentication** (logged in?)
- ✓ **Authorization** (role-based access)
- ✓ **Protecting** unsaved forms

✅ What are Route Guards?

Guards are used to **control access** to routes before navigation happens.

✅ Used in:
AuthGuard

1 CanActivate

- ✓ Controls entry to a route

📁 Example:
only logged-in users
→ can open Dashboard

3 AuthGuard (Common Guard)

- ✓ Implements-CanActivate
- ✓ Checks token/session/role
- ✗ If valid → allow route
- ✗ If invalid → redirect to login

🔥 **Interview line:** ✅ Guards run before navigation
✅ Subject = you can push values anytime → UX.

By Pankaj Ikhar

Solution Architect | Mentor | Coach



Pure vs Impure Pipes

What's the difference?



What is a Pipe?

Pipes transform data in templates without changing original value.

🚩 Example:

```
{{ amount | currency }}  
{{ date | date:"short" }}
```



Pure Pipe (Default)

✓ Runs only when **input value/reference** changes.

✓ Best for **performance**

✓ Used in almost all real apps.



Pure Pipe (Default)

- ✓ Runs only when **input value/reference** changes
- ✓ Best for **performance**
- ✓ Used in almost all real apps.



Impure Pipe (Use Carefully)

- ✓ Runs on **every change** detection cycle.
- ✓ Can slow down app (especially in large lists)
- ✓ Used only when **input** is mutated frequently



Example use-case:

Filtering where array is mutated without new reference.



Interview line: ✓ Pure pipe = fast (runs only on input change)

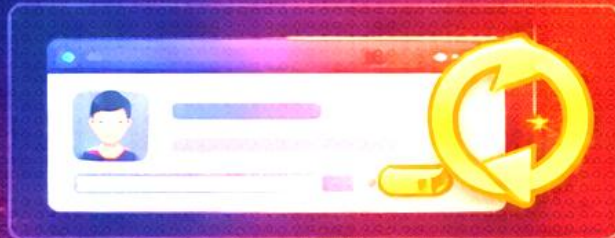


Impure pipe = slower (runs very frequently)

✅ How does Angular Change Detection work?

✅ What is Change Detection?

Angular keeps UI updated by checking when **data changes** and then re-rendering required parts of the UI.



✅ Change Detection Strategies

(Most Asked ⚠️)

✅ Default Strategy

- ✓ Checks most components frequently
- ✓ Easier but can be heavy in large apps.



✅ When does Change Detection run?

Angular automatically triggers change detection when:

🖱️ **User events** ~ click, input, keypress

🌐 **HTTP response** ~ API call results

⌚ **Timers** ~ set Timeout, setInterval

📡 **Observables** ~ emissions

📍 Angular uses, **zone.js** (in most apps) to detect **async** events.

✅ OnPush Strategy

Runs only when:

- ✓ @Input() reference changes
- ✓ Observable emits (async pipe)
- ✓ Component event happens.



🔥 **Interview line:** ✅ Use OnPush + async pipe – immutable updates for faster Angular apps ↗️

By Pankaj Ikhar

✓ Template-driven Forms vs Reactive Forms

✓ Template-driven Forms

- Form logic in the HTML template
- Uses directives: 'ngModel', 'ngForm'
- Creates form model indirectly
- Good for **small/simple forms**

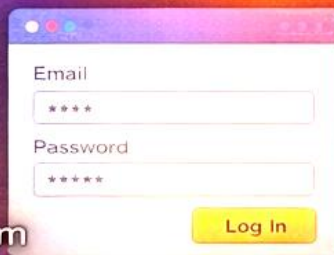
✓ Pros

- ✓ Easy to start
- ✓ Less code in TypeScript

✓ Cons

- ✓ **Hard to scale** for complex forms
- ✗ **Difficult** dynamic forms

📍 **Example:** Login form



A screenshot of a web browser window showing a login form. It has two input fields: 'Email' with four asterisks and 'Password' with six asterisks. Below the fields is a yellow 'Log In' button.

✓ Reactive Forms

- Form logic in TypeScript
- Uses: **FormControl**, **FormGroup**, **FormBuilder**
- Creates form model explicitly
- Good for **large/complex forms**

✓ Pros

- ✓ **More control** + scalable
- ✓ Easier dynamic forms
- ✓ Great for unit testing

✓ Cons

- ✗ **Initial boilerplate code**
- ✗ **Slightly harder to learn**

📍 **Example:** Dynamic form

```
form.addControl() { == }
```



Interview conclusion: Template-driven forms are good for small/simple forms, but **Reactive** forms are preferred for **large, scalable, enterprise apps**. ★

Let's Connect!

For interview preparation, short & long term mentorship, and design & architectural consulting.

 Repost

 Share

 Save

 Comment

-  Job interview preparation
-  Short & long term mentorship
-  Design & architectural consulting

Feel free to reach out:

 greenleaf.educations@gmail.com

 [/in/pankaj-ikhar](https://www.linkedin.com/in/pankaj-ikhar)

 [@Pankaj_ikhar](https://twitter.com/Pankaj_ikhar)

Pankaj Ikhar

Solution Architect & Mentor

★ Mentorship • Interview Prep • Architecture Consulting

